

# GoalWise Product Research and SDLC Blueprint

## Why GoalWise should exist

GoalWise is a strong product concept because it solves a real budgeting failure mode: many people do not struggle only with “tracking expenses,” they struggle with coordinating variable income, fixed obligations, and a near-term savings target under uncertainty. The Federal Reserve’s 2025 Survey of Household Economics and Decisionmaking reported that 30% of adults had income that varied at least occasionally through the year, and income variability was especially high among self-employed adults at 59%. The same Federal Reserve release reported that 59% of adults had at least one major unexpected expense in the prior 12 months, while only 63% said they would cover a \$400 emergency expense with cash or its equivalent. That is exactly the setting where a static monthly category budget often breaks down: money is lumpy, surprises are common, and users need to know what is still safe to spend right now. <sup>1</sup>

GoalWise also fits what budgeting research suggests about behavior. The Consumer Financial Protection Bureau frames budgeting as a tool for getting a handle on debt and “working towards a savings goal.” More importantly for product design, a Journal of Finance study using fintech app data found that setting savings goals increased users’ savings rates, and that the gains were not offset by lower savings outside the app; the paper also highlighted monitoring and concreteness as important mechanisms. Separately, a meta-analysis in *Psychological Bulletin* concluded that progress monitoring is an effective self-regulation strategy that promotes goal attainment, and the National Cancer Institute’s overview of implementation intentions summarizes a large literature showing that “if-then” planning improves goal realization by specifying when, where, and how action should occur. Together, these findings support GoalWise’s core premise: budgeting should be centered on a concrete target, frequent pace feedback, and actionable next steps rather than retrospective category reports alone. <sup>2</sup>

That evidence implies a product principle that should guide the whole SDLC: GoalWise should be designed first as a **goal-attainment system** and only second as an expense tracker. The app’s home screen should answer one question above all others: **“Am I still on pace to hit my target?”** The safest version of the product is therefore not the one with the most categories, but the one that keeps pace calculations trustworthy, explainable, and conservative.

## Where GoalWise fits in the market

GoalWise is differentiated, but it is not entering an empty market. The competitive landscape already contains adjacent ideas. YNAB’s “Targets” help users define how much they want to spend, save, or set aside in specific categories over time. Monarch’s current product combines goal tracking with “Flex Budgeting,” a model that groups spending into Fixed, Non-monthly, and Flex buckets rather than detailed categories, and its Goals feature is built to prioritize financial objectives and suggest adjustments to stay on track. Copilot’s budgeting experience is still organized around a Categories tab and includes features such as budget rebalancing and rollovers when actual spending shifts month to month. PocketSmith goes furthest toward the “one number” concept with Safe Balance, which adjusts an account view to show what is “safe-to-spend” after accounting for planned income and expenses. <sup>3</sup>

That means GoalWise should not position itself as “the first app with goals” or “the first app with safe-to-spend.” The stronger claim is narrower and more defensible: **GoalWise combines goal-first planning, conservative weekly pacing, and automatic recalculation for irregular income in a simpler workflow than category-heavy budgeting tools.** YNAB and Copilot still expect users to reason primarily through categories. Monarch reduces category friction, but its budgeting model still spans multiple budget buckets and multiple goals. PocketSmith’s Safe Balance is account-centric and forecast-centric. GoalWise can own a different mental model: one near-term goal, one target date, one weekly pace number, and one explicit adjustment when reality changes. 4

There is also a favorable adoption context for a product that emphasizes simplicity. A 2025 Financial Technology Association survey found that 79% of consumers said fintech better met their financial needs and gave them a greater sense of control than traditional banks. Meanwhile, fintech infrastructure providers such as Plaid explicitly describe transaction data as foundational for personal finance management, and they redesigned their personal-finance categories to be fewer and better defined for PFM use cases. Those signals suggest that users are comfortable managing money through digital tools, but still benefit when products reduce cognitive load rather than expand it. 5

The implication for your class project is practical: GoalWise should be presented as a **focused consumer fintech product**, not as a giant all-in-one financial operating system. Competing on clarity is more realistic than competing on breadth.

## What GoalWise should do

At product level, GoalWise should let a user enter four things: current cash position, expected income, recurring obligations, and a named goal with a target amount and target date. From there, the app should produce a weekly “safe-to-spend” number, a goal-progress chart, and a pace status such as **ahead, on track, at risk, or off pace**. If the user overspends or earns less than expected, the app should not merely warn them after the fact. It should recalculate the remaining weeks and give three visible choices: reduce weekly spend by a specific amount, extend the target date, or increase planned contributions. That interaction pattern is a direct translation of what the research favors: concreteness, monitoring, and implementation-oriented next steps. 6

A strong MVP should be scoped around one primary use case: **save a defined amount by a defined date without breaking your ability to cover bills.** That leads naturally to six functional requirements. The system should import transactions from CSV or use seeded sample data; let users define recurring income and recurring expenses; let users create one active goal; compute and recompute the weekly safe-to-spend figure; visualize goal progress and pace; and generate plain-English weekly summaries. Category budgets are not the center of the product, but categories still matter for explanations and correction suggestions, so categorization should sit behind the scenes rather than dominate the user experience.

The safest product math is a conservative forecasting model. State and extension guidance for irregular income commonly recommend budgeting from the lowest reliable income month or another conservative baseline instead of the best month. GoalWise should therefore take declared future income and discount it according to a configurable policy for variable earners, such as “confirmed only,” “lowest recent month,” or “conservative rolling average.” That choice is important because your major delivery risk is not interface quality; it is optimistic advice. A conservative cash-flow policy aligns with established guidance for irregular

income and reduces the chance that the app tells users they can spend money that later turns out to be needed. 7

A recommended first-pass formula is:

```
Forecast Resources Until Goal Date
= Current cash
+ Conservative expected income
- Recurring essential expenses due before target date
- Buffer reserve

Goal Gap
= Goal target amount - current amount already saved for goal

Discretionary Capacity
= Forecast Resources Until Goal Date - Goal Gap

Weekly Safe-to-Spend
= max(0, Discretionary Capacity / remaining weeks)
```

That formula should be explainable line by line in the interface. Every recalculation should show the inputs that changed: for example, “You spent \$82 more than your weekly pace, so your remaining weekly safe-to-spend fell from \$214 to \$196,” or “You received an extra stipend deposit, so your weekly safe-to-spend increased by \$18.” The point is not just correctness, but **debuggability for humans**.

Non-functional requirements matter here almost as much as features. For a finance-adjacent tool, the MVP should target accessibility conformant with WCAG 2.2, implement secure-development practices aligned with NIST’s Secure Software Development Framework and OWASP ASVS, and apply privacy-by-design principles such as data minimization from the start. California privacy guidance is particularly relevant for a student team in California: the California Department of Justice notes that consumers have rights to correct inaccurate personal information and limit the use and disclosure of sensitive personal information, while the California Privacy Protection Agency has called data minimization a foundational principle in the CCPA. Even if the classroom prototype does not meet statutory coverage thresholds, designing around these principles is the right discipline for a budgeting app that processes transaction-level data. 8

## How the system should be built

The recommended architecture is deliberately narrow so that the core math remains testable and isolated from AI features.

```
User
↓
React or Next.js frontend
↓
FastAPI application layer
```

- └ Auth and profile service
- └ Goal and recurring-expense service
- └ CSV ingestion and transaction-normalization service
- └ Deterministic pace engine
- └ Rules-based categorization service
- └ Optional LLM summary service

↓

PostgreSQL or SQLite

The **pace engine** is the heart of the system and should be implemented as a pure, deterministic module with versioned formulas and comprehensive unit tests. It should accept normalized inputs and return: forecast resources, goal gap, weekly safe-to-spend, pace status, and an explanation object the frontend can render directly. Because the core value proposition is trust, this service should be independent of the UI and independent of the LLM layer.

The **transaction pipeline** should start simple. Users upload a CSV file or choose seeded transaction data for demo purposes. A normalization step maps merchant text, dates, amounts, and account types into your internal schema. If the team later adds live bank connectivity, Plaid's transaction product and Link flow can support that, but Plaid's own docs make clear that such integrations require token handling and secure storage of access tokens associated with user financial accounts. Deferring live bank sync is therefore the right MVP boundary for a student project. <sup>9</sup>

For **categorization**, GoalWise should prefer layered logic over end-to-end AI. Payment networks already classify merchants using Merchant Category Codes, and Plaid exposes a personal-finance-category taxonomy explicitly designed for PFM apps. That gives you a sensible hierarchy for deterministic mapping when merchants are known. The best design for MVP is: first apply hard rules for recurring obligations and recognized merchants, then use a small classifier or fallback LLM only for ambiguous strings. This reduces hallucination risk and gives the team a measurable accuracy problem instead of an opaque one. <sup>10</sup>

AI should be used, but only in low-authority roles. A good AI boundary for GoalWise is: weekly plain-English summaries, fallback transaction labeling for ambiguous merchant strings, and explanation generation for off-pace situations. A bad AI boundary is: deciding the core weekly spending limit. NIST's AI Risk Management Framework emphasizes govern-map-measure-manage practices for trustworthy AI, which fits this product well: govern the allowed AI use cases, map failure modes such as misleading nudges, measure output quality and drift, and manage human overrides. In product terms, every AI-generated statement should be visibly grounded in the deterministic budget numbers that produced it. <sup>11</sup>

## How to run the SDLC

A strong GoalWise SDLC should look like a disciplined product build rather than a hackathon. NIST explicitly notes that software-security practices need to be integrated into each SDLC implementation, and OWASP ASVS is designed as a basis for testing web-application technical security controls. That combination is a useful backbone for your project plan: security and correctness should be part of discovery, design, coding, testing, and release, not an afterthought. <sup>12</sup>

A practical semester roadmap is below.

| Phase               | Focus                            | Key deliverables                                                                                              |
|---------------------|----------------------------------|---------------------------------------------------------------------------------------------------------------|
| Discovery           | Problem framing, personas, scope | primary persona, user journey, competitor map, top risks, MVP definition                                      |
| Specification       | Product and math design          | user stories, acceptance criteria, pace-engine formula spec, data model, privacy and threat model             |
| Design              | UX and architecture              | wireframes, clickable prototype, API contract, architecture diagram, explanation design                       |
| Build               | Core implementation              | CSV import, recurring-income and expense setup, goal creation, deterministic pace engine, dashboard, charting |
| AI and insights     | Low-risk AI features             | weekly summary prompts, ambiguity fallback for transactions, human-review rules                               |
| Test and harden     | Quality gates                    | unit tests for pace math, integration tests, security checklist, accessibility audit, usability walkthrough   |
| Release and operate | Demo-ready deployment            | staging deployment, seed dataset, admin scripts, runbook, known issues log, retrospective                     |

For team roles, a four-person structure works well. One person should own product and project management, backlog, and acceptance criteria. One should own backend services and the pace engine. One should own frontend and UX, especially the explanation and charting flows. One should own QA, test automation, documentation, and release. AI prompt design can sit with backend or QA, but the prompts should live in source control and be evaluated against a labeled dataset of expected outputs rather than treated as magic strings.

Testing should be unusually rigorous around the finance logic. The pace engine needs golden test cases for steady income, irregular income, early overspend, under-spend, unexpected bills, date extensions, and goal completion. The expectation should be near-complete coverage of calculation branches, because this module is the promise of the product. Integration tests should cover CSV ingestion, recurring rules, and database persistence. Manual usability testing should verify that real users can understand why a weekly number changed without staff explanation. Accessibility testing should include keyboard-only navigation, screen-reader labeling, contrast checks, and error-state clarity in line with WCAG 2.2. <sup>13</sup>

The release strategy should also reflect the product's risk profile. A sensible rollout is a closed pilot with sample or synthetic data first, then optional user CSV import, and only later live financial account linking if the team chooses to extend the project. That sequencing matches both secure-development practice and the reality that tokenized account aggregation introduces more security, privacy, and compliance burden than a class MVP needs. <sup>14</sup>

## Risk, compliance, and success measures

The single most serious delivery and product risk remains the one you identified: an overly optimistic safe-to-spend number could tell users they have more flexibility than they really do. That is not a generic “accuracy” risk. It is existential, because a budgeting tool that causes missed goals or missed bills breaks

trust faster than almost any UI bug. The right mitigation stack is four layers deep. First, GoalWise should use a conservative income policy for variable earners. Second, it should maintain an explicit buffer reserve. Third, it should recalculate at weekly cadence or faster whenever new spending lands. Fourth, it should expose the full formula and assumptions instead of presenting a black-box number. Those choices align with irregular-income guidance that emphasizes conservative baselines and with the broader self-regulation literature showing that repeated monitoring and specific plans improve goal performance. <sup>15</sup>

There are two important secondary risks. One is **AI overreach**: if the summary layer sounds authoritative when it is only probabilistic, users may treat nudge text as financial advice. The mitigation is to keep AI out of the limit calculation, label summaries as explanatory, and tie every nudge to visible deterministic numbers. The other is **privacy creep**: budgeting products are naturally tempted to retain ever more financial detail, but California regulators have specifically emphasized data minimization as foundational. GoalWise should therefore store only what the MVP needs, provide deletion and correction paths in the product design, and avoid collecting bank credentials in the prototype altogether. <sup>16</sup>

If you want measurable success criteria for the project, use a mix of product, quality, and trust metrics. Product metrics should include time-to-first-goal, percentage of users who complete setup, weekly active usage, and percentage of users still “on pace” after four weeks in test scenarios. Quality metrics should include pace-engine test pass rate, categorization accuracy on a labeled CSV dataset, accessibility findings resolved, and recalculation latency. Trust metrics should include percentage of explanations users say they understand, percentage of weekly-limit changes that are traceable in the UI, and zero uncovered high-severity security findings against your chosen ASVS checklist. <sup>17</sup>

The overall recommendation is clear: GoalWise is a credible, differentiated fintech project if you treat it as a **trustworthy pacing engine with a simple interface**, not as a general-purpose budgeting app. The research supports goal-setting, monitoring, and concreteness; the market leaves room for a simpler, one-outcome budgeting experience; and the SDLC can be scoped cleanly around a deterministic core with optional AI on the edges. If the team protects that center of gravity throughout design, testing, and release, GoalWise has the ingredients of both a strong class project and a plausible real product.

---

<sup>1</sup> The Fed - Income and Expenses

[https://www.federalreserve.gov/publications/2026-economic-well-being-of-us-households-in-2025-Income-and-Expenses.htm?utm\\_source=chatgpt.com](https://www.federalreserve.gov/publications/2026-economic-well-being-of-us-households-in-2025-Income-and-Expenses.htm?utm_source=chatgpt.com)

<sup>2</sup> Budgeting: How to create a budget and stick with it

[https://www.consumerfinance.gov/archive/blog/budgeting-how-to-create-a-budget-and-stick-with-it/?utm\\_source=chatgpt.com](https://www.consumerfinance.gov/archive/blog/budgeting-how-to-create-a-budget-and-stick-with-it/?utm_source=chatgpt.com)

<sup>3</sup> <sup>4</sup> How to Use Targets

[https://support.ynab.com/en\\_us/how-to-use-targets-rk5kki9ks?utm\\_source=chatgpt.com](https://support.ynab.com/en_us/how-to-use-targets-rk5kki9ks?utm_source=chatgpt.com)

<sup>5</sup> State of Fintech Survey

[https://www.ftassociation.org/wp-content/uploads/2025/06/FTA-State-of-Fintech-Survey.pdf?utm\\_source=chatgpt.com](https://www.ftassociation.org/wp-content/uploads/2025/06/FTA-State-of-Fintech-Survey.pdf?utm_source=chatgpt.com)

<sup>6</sup> Goal Setting and Saving in the FinTech Era

[https://ideas.repec.org/a/bla/jfinan/v79y2024i3p1931-1976.html?utm\\_source=chatgpt.com](https://ideas.repec.org/a/bla/jfinan/v79y2024i3p1931-1976.html?utm_source=chatgpt.com)

<sup>7</sup> <sup>15</sup> How to Budget Effectively with an Irregular Income

[https://ndbf.nebraska.gov/how-budget-effectively-irregular-income?utm\\_source=chatgpt.com](https://ndbf.nebraska.gov/how-budget-effectively-irregular-income?utm_source=chatgpt.com)

8 13 WCAG 2 Overview | Web Accessibility Initiative (WAI)

[https://www.w3.org/WAI/standards-guidelines/wcag/?utm\\_source=chatgpt.com](https://www.w3.org/WAI/standards-guidelines/wcag/?utm_source=chatgpt.com)

9 Introduction to Transactions | Plaid Docs

[https://plaid.com/docs/transactions/?utm\\_source=chatgpt.com](https://plaid.com/docs/transactions/?utm_source=chatgpt.com)

10 Payments - Merchant Category Code (MCC)

[https://support.visaacceptance.com/knowledgebase/knowledgearticle/?code=KA-09260&utm\\_source=chatgpt.com](https://support.visaacceptance.com/knowledgebase/knowledgearticle/?code=KA-09260&utm_source=chatgpt.com)

11 16 AI Risk Management Framework | NIST

[https://www.nist.gov/itl/ai-risk-management-framework?utm\\_source=chatgpt.com](https://www.nist.gov/itl/ai-risk-management-framework?utm_source=chatgpt.com)

12 14 Secure Software Development Framework | CSRC

[https://csrc.nist.gov/projects/ssdf?utm\\_source=chatgpt.com](https://csrc.nist.gov/projects/ssdf?utm_source=chatgpt.com)

17 OWASP Application Security Verification Standard (ASVS)

[https://owasp.org/www-project-application-security-verification-standard/?utm\\_source=chatgpt.com](https://owasp.org/www-project-application-security-verification-standard/?utm_source=chatgpt.com)